

SCBX - QAOA Report

Andrew — Naravit Namson

Contents

Report 1 — June 18, 2025	3
Setup	3
Problem	3
Method	3
1. Normalize for Discrete Version	3
2. Reformulate Constraint to Quadratic Programming	4
3. Generate QUBO and Ising Form	4
4. Translate QUBO to Ising Hamiltonian	5
5. Programming	5
Result	6
Distribution Result: No Risk Case ($q = 0$)	6
1 Methodology	8
1.1 Data Preparation	8
1.2 Problem Formulation	8
1.3 Normalization and Binary Encoding	8
1.3.1 Price Normalization	9
1.3.2 Binary Encoding	9
1.4 QUBO Formulation	9
1.4.1 Hyperparameter Calibration of q and λ	10
1.5 Ising Hamiltonian	10
1.6 QAOA Circuit	11
1.6.1 Problem Unitary	11
1.6.2 Mixer Unitary: X-Mixer (Standard)	11
1.6.3 Mixer Unitary: Constraint-Preserving Mixer	11
1.7 Parameter Optimization	14
1.7.1 CUDAQ Built-in Optimizers	14
1.7.2 Adam Optimization with Cosine Annealing	14
1.7.3 Parameter Initialization	15
1.8 Solution Evaluation	15
1.8.1 Feasible Set Definition	15
1.8.2 Approximation Ratio	15
1.8.3 Max-Probability Ratio	15

1.8.4	Additional Metrics	15
1.9	Barren Plateau Analysis	16
1.10	Experimental Configuration	16

Report 1 — June 18, 2025

Setup

For N assets indexed by $i \in \{1, \dots, N\}$ observed over $T + 1$ days indexed by $t \in \{0, \dots, T\}$, let p_i^t denote the closing price of asset i on day t .

We define the return ratio of asset i at day t as:

$$r_i^t = \frac{p_i^t - p_i^{t-1}}{p_i^{t-1}} \quad \text{for } t = 1, \dots, T \quad (1)$$

The average return vector $\mu \in \mathbb{R}^N$ is given by:

$$\mu_i = \mathbb{E}[r_i] = \frac{1}{T} \sum_{t=1}^T r_i^t \quad (2)$$

The covariance matrix $\Sigma = [\sigma_{ij}] \in \mathbb{R}^{N \times N}$ is defined as:

$$\sigma_{ij} = \mathbb{E}[(r_i - \mu_i)(r_j - \mu_j)] = \frac{1}{T-1} \sum_{t=1}^T (r_i^t - \mu_i)(r_j^t - \mu_j) \quad (3)$$

Let the current price of asset i be $P_i = p_i^T$, and define the price vector at time T as $\mathbf{P} = [P_1, P_2, \dots, P_N]^T$.

Suppose we are given a total budget $B \in \mathbb{R}$.

Problem

We seek a portfolio allocation vector $\mathbf{x} \in \mathbb{R}^N$ that maximizes the trade-off between expected return and risk.

The optimization problem is:

$$\begin{aligned} \max_{\mathbf{x}} \quad & (\mu^T \mathbf{x} - q \cdot \mathbf{x}^T \Sigma \mathbf{x}) \\ \text{s.t.} \quad & \mathbf{P}^T \mathbf{x} = B \end{aligned} \quad (4)$$

Here, $q \in \mathbb{R}$ is a tunable hyperparameter that controls the trade-off between expected return and portfolio risk. The term $\mu^T \mathbf{x}$ represents the expected return, while $\mathbf{x}^T \Sigma \mathbf{x}$ corresponds to the portfolio variance.

Method

****Formulation based on this paper.****

1. Normalize Variables for Discrete Version

We normalize the portfolio optimization problem for discrete encoding as follows.

Let

$$P' = \frac{P}{B}, \quad \mu' = \text{diag}(P')\mu, \quad \Sigma' = \text{diag}(P')\Sigma \text{diag}(P') \quad (5)$$

where

$$\text{diag}([v_1, v_2, v_3, \dots]) := \begin{bmatrix} v_1 & 0 & 0 & \dots \\ 0 & v_2 & 0 & \dots \\ 0 & 0 & v_3 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

For each asset i , define d_i as the largest non-negative integer such that $2^{d_i} \leq \frac{B}{P_i}$, i.e., the maximum exponent of 2 allowed by the budget:

$$d_i = \left\lfloor \log_2 \left(\frac{B}{P_i} \right) \right\rfloor \quad (6)$$

Let the total number of required binary variables (qubits) be:

$$\text{qb} = \sum_{i=1}^N (d_i + 1) \quad (7)$$

Construct the binary encoding matrix $C \in \mathbb{R}^{N \times \text{qb}}$ as:

$$C = \begin{pmatrix} 2^0 & \dots & 2^{d_1} & 0 & \dots & 0 & \dots & 0 & \dots & 0 \\ 0 & \dots & 0 & 2^0 & \dots & 2^{d_2} & \dots & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 0 & \dots & 0 & \dots & 2^0 & \dots & 2^{d_N} \end{pmatrix} \quad (8)$$

Then the quantized variables are:

$$P'' = C^T P', \quad \mu'' = C^T \mu', \quad \Sigma'' = C^T \Sigma' C \quad (9)$$

The optimization problem becomes:

$$\begin{aligned} \max_{\mathbf{b}} \quad & \mu''^T \mathbf{b} - q \cdot \mathbf{b}^T \Sigma'' \mathbf{b} \\ \text{s.t.} \quad & \mathbf{P}''^T \mathbf{b} = 1 \\ & \mathbf{b} \in \{0, 1\}^{\text{qb}} \end{aligned} \quad (10)$$

2. Reformulate Constraint to Quadratic Programming

To convert the problem into unconstrained quadratic form, we add a penalty term with hyperparameter $\lambda \in \mathbb{R}$ to enforce the budget constraint:

$$\max_{\mathbf{b}} \quad \mu''^T \mathbf{b} - q \cdot \mathbf{b}^T \Sigma'' \mathbf{b} - \lambda (\mathbf{P}''^T \mathbf{b} - 1)^2 \quad (11)$$

3. Generate QUBO and Ising Form

We rewrite the constraint as a penalty term:

$$(\mathbf{P}''^T \mathbf{b} - 1)^2 = \left(\sum_i P_i'' b_i - 1 \right)^2 \quad (12)$$

$$= \sum_i P_i''^2 b_i + 2 \sum_{i \neq j} P_i'' P_j'' b_i b_j - 2 \sum_i P_i'' b_i + 1 \quad (13)$$

$$= - \sum_i P_i''^2 b_i + 2 \sum_{i,j} P_i'' P_j'' b_i b_j - 2 \sum_i P_i'' b_i + 1 \quad (14)$$

Substituting the expansion from Equation (14) into the objective in Equation (11), we obtain:

$$\max_{\mathbf{b}} \sum_i b_i (\mu_i'' + \lambda(P_i'' + 2P_i'')) - \sum_{i,j} b_i b_j (q \cdot \Sigma_{ij}'' + 2\lambda P_i'' P_j'') \quad (15)$$

We define the QUBO matrix $Q \in \mathbb{R}^{\text{qb} \times \text{qb}}$ as:

$$Q = \text{diag}(\mu'' + 2\lambda P'') + \lambda \cdot \text{diag}(P'')^2 - q \cdot \Sigma'' - 2\lambda P'' P''^T \quad (16)$$

Then the final QUBO formulation is:

$$\max_{\mathbf{b} \in \{0,1\}^{\text{qb}}} \mathbf{b}^T Q \mathbf{b} \quad (17)$$

4. Translate QUBO to Ising Hamiltonian

To prepare for QAOA, we convert the QUBO into an Ising Hamiltonian. Since QAOA minimizes the energy, we negate the QUBO objective:

$$H = - \sum_{i,j} Q_{ij} \cdot \frac{(I - Z_i)}{2} \otimes \frac{(I - Z_j)}{2} \quad (18)$$

Where Z_i is the Pauli-Z operator on qubit i .

5. Programming

We implemented the QAOA procedure using **CUDA-Q** with the Python API, running on a local GPU setup. Both gradient-based (Adam, GradientDescent) and non-gradient-based (COBYLA) optimizers were explored.

We also investigated the choice of hyperparameters:

- $\lambda \in \mathbb{R}$: to enforce the budget constraint
- $q \in \mathbb{R}$: to balance expected return and risk

For the binary encoding, each asset i uses $d_i + 1$ qubits to represent its discrete investment level. For example, if the first asset uses 2 qubits, its quantity is represented as the binary number $q_1 q_0$; if the second asset uses 3 qubits, it is encoded as $q_5 q_4 q_3$.

The full binary vector $\mathbf{b} \in \{0,1\}^{\text{qb}}$ encodes the entire portfolio as a simple concatenation $q_0 q_1 \dots q_5$, which is easy to implement. In future versions, this structure can be refactored for improved readability and modularity.

Result

Both **Adam** and **COBYLA** provided favorable trade-offs:

- **COBYLA** converged faster in early iterations.
- **Adam** achieved lower final energy and showed smoother convergence, as illustrated in Fig. 1.

For larger problem sizes (10 qubits and above), we found that plain **GradientDescent**—without adaptive learning rate or momentum—performed poorly (as shown in Fig. 1). This may be attributed to the highly rugged loss landscape introduced by the budget constraint, as discussed in this paper shared by Prof. Pipe.

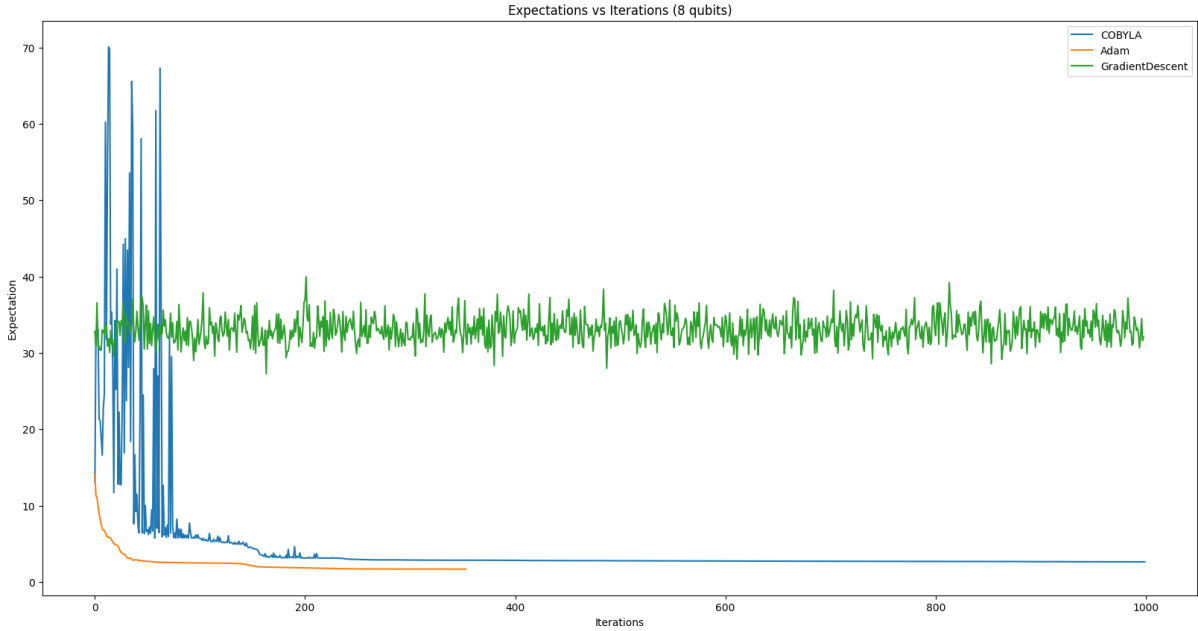


Figure 1: Expectation vs. Iteration for 8-qubit problem using different optimizers.

Additionally, we observed that for larger budget values, it is necessary to increase λ to maintain a high probability of staying within the feasible region. This is because the magnitude of the return term grows with budget, requiring stronger constraint enforcement.

Distribution Result: No Risk Case ($q = 0$)

For the setup $B = 270$, $P = [195, 183, 131]$, $\mu = [0.0011, 0.0008, 0.0007]$, with $\lambda = 3$ and $q = 0$ (ignoring risk), each asset uses 1, 1, and 2 qubits respectively.

The optimal solution is to buy 2 stocks of the last asset, which gives the best return and nearly exhausts the budget. This is encoded as $0|0|01 = 1$ (in base 10).

The output distribution from QAOA (executed using CUDA-Q) is shown in Fig. 2.

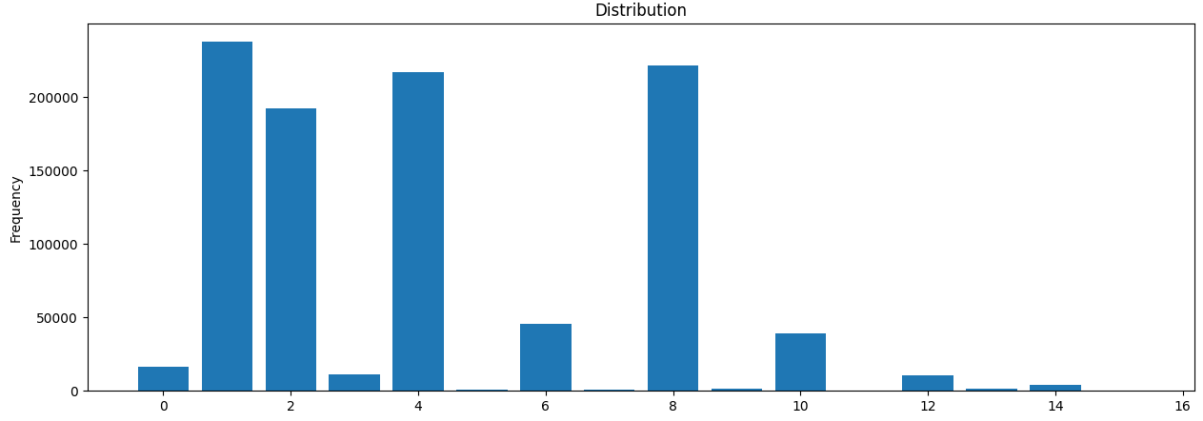


Figure 2: Distribution of measurement outcomes for QAOA with $q = 0$ (risk ignored).

Each bar represents a binary portfolio state, interpreted as:

- $0 = 0|0|00 \rightarrow$ buy nothing
- $1 = 0|0|01 \rightarrow$ buy 2 of last asset
- $2 = 0|0|10 \rightarrow$ buy 1 of last asset
- $3 = 0|0|11 \rightarrow$ buy 3 of last asset
- $4 = 0|1|00 \rightarrow$ buy 1 of middle asset
- $6 = 0|1|10 \rightarrow$ buy 1 each of middle and last
- $8 = 1|0|00 \rightarrow$ buy 1 of first asset
- $10 = 1|0|10 \rightarrow$ buy 1 each of first and last
- $12 = 1|1|00 \rightarrow$ buy 1 each of first and middle
- $14 = 1|1|10 \rightarrow$ buy 1 each of all assets

1 Methodology

This section presents the complete methodology for solving the portfolio optimization problem using the Quantum Approximate Optimization Algorithm (QAOA) [1]. The pipeline proceeds through seven stages: data preparation, problem normalization, QUBO formulation, Ising Hamiltonian construction, QAOA circuit design (with two mixer variants), parameter optimization, and solution evaluation.

1.1 Data Preparation

We consider a universe of N_{total} assets observed over $T + 1$ trading days indexed by $t \in \{0, \dots, T\}$. Let p_i^t denote the closing price of asset i on day t .

The daily return ratio of asset i at day t is:

$$r_i^t = \frac{p_i^t - p_i^{t-1}}{p_i^{t-1}}, \quad t = 1, \dots, T \quad (19)$$

The expected return vector $\boldsymbol{\mu} \in \mathbb{R}^N$ and covariance matrix $\Sigma \in \mathbb{R}^{N \times N}$ are estimated as:

$$\mu_i = \frac{1}{T} \sum_{t=1}^T r_i^t, \quad \sigma_{ij} = \frac{1}{T-1} \sum_{t=1}^T (r_i^t - \mu_i)(r_j^t - \mu_j) \quad (20)$$

The current price of each asset is taken as its last observed closing price $P_i = p_i^T$, forming the price vector $\mathbf{P} = [P_1, \dots, P_N]^\top$.

The dataset consists of $N_{\text{total}} = 50$ US equities. For each experiment, a subset of N assets is randomly selected from the universe. The assets are filtered by a price range $[P_{\min}, P_{\max}]$ to ensure comparable qubit encodings across assets. Each experiment uses a deterministic random seed derived from the experiment index and asset count, ensuring full reproducibility.

1.2 Problem Formulation

Following the classical mean-variance framework [6], given a total budget $B \in \mathbb{R}$, we seek a portfolio allocation vector $\mathbf{x} \in \mathbb{R}^N$ that maximizes the trade-off between expected return and risk:

$$\begin{aligned} \max_{\mathbf{x}} \quad & \boldsymbol{\mu}^\top \mathbf{x} - q \cdot \mathbf{x}^\top \Sigma \mathbf{x} \\ \text{s.t.} \quad & \mathbf{P}^\top \mathbf{x} = B \end{aligned} \quad (21)$$

where $q \geq 0$ is a risk-aversion parameter controlling the trade-off between expected return $\boldsymbol{\mu}^\top \mathbf{x}$ and portfolio variance $\mathbf{x}^\top \Sigma \mathbf{x}$.

1.3 Normalization and Binary Encoding

To map the continuous problem to a binary (qubit) representation, we normalize all quantities by the budget and encode asset quantities in binary.

1.3.1 Price Normalization

Define the normalized price, return, and covariance:

$$\mathbf{P}' = \frac{\mathbf{P}}{B}, \quad \boldsymbol{\mu}' = \text{diag}(\mathbf{P}') \boldsymbol{\mu}, \quad \Sigma' = \text{diag}(\mathbf{P}') \Sigma \text{diag}(\mathbf{P}') \quad (22)$$

After normalization, the budget constraint becomes $\mathbf{P}'^\top \mathbf{x} = 1$.

1.3.2 Binary Encoding

For each asset i , the maximum number of units purchasable is $\lfloor B/P_i \rfloor$. We encode the quantity of asset i using $d_i + 1$ binary variables (qubits), where:

$$d_i = \left\lceil \log_2 \left(\frac{B}{P_i} \right) \right\rceil \quad (23)$$

The total number of qubits required is:

$$n = \sum_{i=1}^N (d_i + 1) \quad (24)$$

We construct the binary encoding matrix $C \in \mathbb{R}^{N \times n}$:

$$C = \begin{pmatrix} 2^0 & \dots & 2^{d_1} & 0 & \dots & 0 & \dots & 0 & \dots & 0 \\ 0 & \dots & 0 & 2^0 & \dots & 2^{d_2} & \dots & 0 & \dots & 0 \\ \vdots & & \vdots & \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \dots & 0 & 0 & \dots & 0 & \dots & 2^0 & \dots & 2^{d_N} \end{pmatrix} \quad (25)$$

The relationship between the continuous allocation $\mathbf{x}$ and the binary vector $\mathbf{b} \in \{0, 1\}^n$ is $\mathbf{x} = C\mathbf{b}$. The quantized problem variables become:

$$\mathbf{P}'' = C^\top \mathbf{P}', \quad \boldsymbol{\mu}'' = C^\top \boldsymbol{\mu}', \quad \Sigma'' = C^\top \Sigma' C \quad (26)$$

The discrete optimization problem is:

$$\begin{aligned} \max_{\mathbf{b}} \quad & \boldsymbol{\mu}''^\top \mathbf{b} - q \cdot \mathbf{b}^\top \Sigma'' \mathbf{b} \\ \text{s.t.} \quad & \mathbf{P}''^\top \mathbf{b} = 1, \quad \mathbf{b} \in \{0, 1\}^n \end{aligned} \quad (27)$$

1.4 QUBO Formulation

To remove the equality constraint, we add a quadratic penalty term with hyperparameter $\lambda > 0$:

$$\max_{\mathbf{b} \in \{0, 1\}^n} \quad \boldsymbol{\mu}''^\top \mathbf{b} - q \cdot \mathbf{b}^\top \Sigma'' \mathbf{b} - \lambda (\mathbf{P}''^\top \mathbf{b} - 1)^2 \quad (28)$$

Expanding the penalty term using $b_i^2 = b_i$ for binary variables:

$$(\mathbf{P}''^\top \mathbf{b} - 1)^2 = \sum_{i,j} P_i'' P_j'' b_i b_j - 2 \sum_i P_i'' b_i + 1 \quad (29)$$

We define the QUBO matrix $Q \in \mathbb{R}^{n \times n}$:

$$Q = \text{diag}(\boldsymbol{\mu}'' + 2\lambda \mathbf{P}'') - \lambda \mathbf{P}'' \mathbf{P}''^\top - q \Sigma'' \quad (30)$$

so that the problem becomes:

$$\max_{\mathbf{b} \in \{0,1\}^n} \mathbf{b}^\top Q \mathbf{b} \quad (31)$$

Note: the constant terms $-\lambda$ are absorbed into the Hamiltonian offset and do not affect the optimization.

1.4.1 Hyperparameter Calibration of q and λ

In Equation 28, the hyperparameters q and λ control the relative weight of the three competing terms: the expected return $\boldsymbol{\mu}''^\top \mathbf{b}$, the risk penalty $q \cdot \mathbf{b}^\top \Sigma'' \mathbf{b}$, and the budget penalty $\lambda(\mathbf{P}''^\top \mathbf{b} - 1)^2$. If q or λ is too large, the corresponding penalty dominates the cost landscape and the optimizer ignores the return signal; if too small, the constraint is not enforced.

To calibrate these values, we construct the Ising Hamiltonian $H_{\text{problem}}(q, \lambda)$ corresponding to Equation 28 and prepare the uniform superposition $|\psi\rangle = H^{\otimes n}|0\rangle^{\otimes n}$. We then evaluate $\langle\psi|H_{\text{problem}}(q, \lambda)|\psi\rangle$ under three settings to isolate each term:

1. $(q, \lambda) = (0, 0)$: measures the expected return magnitude $\langle\psi|H_{\text{ret}}|\psi\rangle$.
2. $(q, \lambda) = (q_{\text{large}}, 0)$: measures the risk penalty magnitude $\langle\psi|H_{\text{risk}}|\psi\rangle$.
3. $(q, \lambda) = (0, \lambda_{\text{large}})$: measures the budget penalty magnitude $\langle\psi|H_{\text{budget}}|\psi\rangle$.

By comparing these three magnitudes and selecting q and λ such that the terms are of comparable scale, we obtain $\lambda = 0.0005$ and $q = 1.5$.

1.5 Ising Hamiltonian

To map the QUBO to a quantum Hamiltonian, we substitute each binary variable with a Pauli-Z operator via $b_i = \frac{I - Z_i}{2}$, yielding the cost Hamiltonian:

$$H_C = - \sum_{i,j} Q_{ij} \cdot \frac{(I - Z_i)}{2} \cdot \frac{(I - Z_j)}{2} + \lambda I \quad (32)$$

The ground state of H_C corresponds to the optimal portfolio allocation.

After expansion and simplification, H_C decomposes into single-qubit terms with coefficients $\{h_k\}$ and two-qubit interaction terms with coefficients $\{J_{kl}\}$:

$$H_C = \text{const} + \sum_k h_k Z_k + \sum_{k < l} J_{kl} Z_k Z_l \quad (33)$$

In practice, the Hamiltonian is multiplied by a boost factor $\alpha > 0$ (e.g., $\alpha = 7500$) to improve numerical stability during optimization:

$$H_{\text{ansatz}} = \alpha \cdot H_C \quad (34)$$

1.6 QAOA Circuit

The Quantum Approximate Optimization Algorithm (QAOA) [1] constructs a parameterized quantum circuit with p layers. Each layer applies a problem unitary $U_P(\gamma)$ followed by a mixer unitary $U_M(\beta)$. The circuit output state is:

$$|\gamma, \beta\rangle = \prod_{l=1}^p U_M(\beta_l) U_P(\gamma_l) |\psi_0\rangle \quad (35)$$

where $\gamma = (\gamma_1, \dots, \gamma_p)$ and $\beta = (\beta_1, \dots, \beta_p)$ are the variational parameters, giving $2p$ total parameters.

1.6.1 Problem Unitary

The problem unitary encodes the cost Hamiltonian. For single-qubit terms with coefficient h_k :

$$U_{P,k}^{(1)}(\gamma) = R_Z(2 h_k \gamma, k) = e^{-i h_k \gamma Z_k} \quad (36)$$

For two-qubit interaction terms with coefficient J_{kl} :

$$U_{P,kl}^{(2)}(\gamma) = \text{CNOT}_{k,l} \cdot R_Z(2 J_{kl} \gamma, l) \cdot \text{CNOT}_{k,l} \quad (37)$$

This implements $e^{-i J_{kl} \gamma Z_k Z_l}$ using the standard CNOT- R_Z -CNOT decomposition. The full problem unitary for layer l is:

$$U_P(\gamma_l) = \prod_{k < l'} U_{P,kl'}^{(2)}(\gamma_l) \prod_k U_{P,k}^{(1)}(\gamma_l) \quad (38)$$

1.6.2 Mixer Unitary: X-Mixer (Standard)

The standard X-mixer applies a transverse-field rotation to all qubits:

$$U_M^{(X)}(\beta) = \prod_{k=0}^{n-1} R_X(2\beta, k) = \prod_{k=0}^{n-1} e^{-i \beta X_k} \quad (39)$$

The initial state is the uniform superposition:

$$|\psi_0\rangle = |+\rangle^{\otimes n} = H^{\otimes n} |0\rangle^{\otimes n} \quad (40)$$

The X-mixer explores the full 2^n -dimensional Hilbert space, including states that violate the budget constraint. The budget constraint is enforced only through the penalty term in H_C with strength λ .

1.6.3 Mixer Unitary: Constraint-Preserving Mixer

The constraint-preserving mixer [3] restricts the quantum evolution to a subspace of near-feasible states, eliminating the need for the penalty term in the cost Hamiltonian.

Feasible State Selection. We define a feasibility tolerance $\varepsilon > 0$ and compute the budget violation for every computational basis state $|\mathbf{b}\rangle$:

$$\Delta(\mathbf{b}) = |\mathbf{P}^{\prime\prime\top} \mathbf{b} - 1| \quad (41)$$

The feasible set is:

$$\mathcal{F}_\varepsilon = \{\mathbf{b} \in \{0, 1\}^n \mid \Delta(\mathbf{b}) \leq \varepsilon\} \quad (42)$$

From $\mathcal{F}_\varepsilon$, we select the K states with the smallest budget violation. Two strategies are available:

(a) Brute-force enumeration. All 2^n computational basis states are evaluated, sorted by $\Delta(\mathbf{b})$, and the K states with the smallest violation are selected. This approach yields the exact optimum but scales as $O(2^n)$, making it infeasible for large qubit counts.

(b) Genetic Algorithm (GA). For scalability, we employ a Genetic Algorithm that searches for near-feasible states without enumerating the full Hilbert space. Each individual's chromosome encodes an n -bit binary string $\mathbf{b}$, and the fitness function is defined as the negative budget violation $-\Delta(\mathbf{b})$ (i.e., lower violation yields higher fitness). The GA operates with the following hyperparameters:

- **Population size:** $N_{\text{pop}} = 2000$
- **Generations:** $G = 35$
- **Crossover rate:** $p_c = 0.85$
- **Mutation rate:** $p_m = 1.5/n$, scaled inversely with chromosome length
- **Selection:** tournament selection with tournament size 5
- **Elitism:** top 2 individuals are preserved across generations

After G generations, the top K individuals (ranked by fitness) are extracted as the basis states. The GA scales polynomially with n , enabling feasible state selection for problem sizes where brute-force enumeration is intractable.

These K selected states form the basis set $\mathcal{B} = \{\mathbf{b}^{(1)}, \dots, \mathbf{b}^{(K)}\}$.

Initial State. The initial state is a uniform superposition over the selected basis states:

$$|\psi_0\rangle = \frac{1}{\sqrt{K}} \sum_{k=1}^K |\mathbf{b}^{(k)}\rangle \quad (43)$$

Transition Graph. We define a cyclic transition graph over the K basis states with adjacency matrix $T \in \mathbb{R}^{K \times K}$:

$$T_{ij} = \begin{cases} 1 & \text{if } |i - j| = 1 \pmod{K} \\ 0 & \text{otherwise} \end{cases} \quad (44)$$

This forms a ring topology connecting each basis state to its two neighbors.

Mixer Hamiltonian Construction. For each pair of connected basis states $(\mathbf{b}^{(i)}, \mathbf{b}^{(j)})$ with $T_{ij} = 1$, we construct the transition operator $|\mathbf{b}^{(i)}\rangle\langle\mathbf{b}^{(j)}| + |\mathbf{b}^{(j)}\rangle\langle\mathbf{b}^{(i)}|$ and decompose it into a sum of Pauli strings following the recursive construction of [2].

Let $|z\rangle$ and $|w\rangle$ be two n -qubit computational basis states. We decompose $|z\rangle\langle w| + |w\rangle\langle z|$ into Pauli strings recursively. Define two operators A_n (the real/Hermitian part) and B_n (the imaginary/anti-Hermitian part):

$$A_n = |z\rangle\langle w| + |w\rangle\langle z|, \quad B_n = i(|z\rangle\langle w| - |w\rangle\langle z|) \quad (45)$$

For a single qubit ($n = 1$), the base cases are:

$$A_1 = \begin{cases} I + Z, & \text{if } (z, w) = (0, 0), \\ X, & \text{if } (z, w) = (0, 1), \\ X, & \text{if } (z, w) = (1, 0), \\ I - Z, & \text{if } (z, w) = (1, 1), \end{cases} \quad B_1 = \begin{cases} 0, & \text{if } (z, w) = (0, 0), \\ -Y, & \text{if } (z, w) = (0, 1), \\ +Y, & \text{if } (z, w) = (1, 0), \\ 0, & \text{if } (z, w) = (1, 1). \end{cases} \quad (46)$$

The recursive step from n to $n + 1$ qubits, with the $(n + 1)$ -th qubit having bits (x, y) , is:

$$A_{n+1} = \frac{1}{2} \begin{cases} A_n \otimes (I + Z), & \text{if } (x, y) = (0, 0), \\ A_n \otimes X + B_n \otimes Y, & \text{if } (x, y) = (0, 1), \\ A_n \otimes X - B_n \otimes Y, & \text{if } (x, y) = (1, 0), \\ A_n \otimes (I - Z), & \text{if } (x, y) = (1, 1), \end{cases} \quad (47)$$

$$B_{n+1} = \frac{1}{2} \begin{cases} B_n \otimes (I + Z), & \text{if } (x, y) = (0, 0), \\ B_n \otimes X - A_n \otimes Y, & \text{if } (x, y) = (0, 1), \\ B_n \otimes X + A_n \otimes Y, & \text{if } (x, y) = (1, 0), \\ B_n \otimes (I - Z), & \text{if } (x, y) = (1, 1). \end{cases} \quad (48)$$

The transition operator is the final A_n after processing all n qubits. The mixer Hamiltonian is:

$$H_M = \sum_{\substack{i < j \\ T_{ij} \neq 0}} T_{ij} \sum_m c_m \hat{P}_m \quad (49)$$

where $\hat{P}_m$ are Pauli strings and c_m are real coefficients obtained from the decomposition.

The preserving mixer unitary is:

$$U_M^{(\text{Pres})}(\beta) = \prod_m e^{-i c_m \beta \hat{P}_m} \quad (50)$$

Since this mixer preserves the feasible subspace by construction, the cost Hamiltonian for the preserving mixer uses $\lambda = 0$ (no budget penalty):

$$H_{\text{ansatz}}^{(\text{Pres})} = \alpha \cdot \left(- \sum_{i,j} Q_{ij}^{(\text{eval})} \cdot \frac{(I - Z_i)}{2} \cdot \frac{(I - Z_j)}{2} \right) \quad (51)$$

where $Q^{(\text{eval})}$ is the QUBO matrix with $\lambda = 0$.

1.7 Parameter Optimization

The variational parameters (γ, β) are optimized to minimize the expectation value of H_{ansatz} :

$$(\gamma^*, \beta^*) = \arg \min_{\gamma, \beta} \langle \gamma, \beta | H_{\text{ansatz}} | \gamma, \beta \rangle \quad (52)$$

Two optimization strategies are employed:

1.7.1 CUDAQ Built-in Optimizers

Classical optimizers from the CUDAQ library, including COBYLA (gradient-free) and Adam (gradient-based with forward-difference gradient estimation):

$$\frac{\partial \langle H \rangle}{\partial \theta_j} \approx \frac{\langle H \rangle_{\theta_j + \delta} - \langle H \rangle_{\theta_j}}{\delta} \quad (53)$$

where δ is a small shift (typically 10^{-4}).

1.7.2 Adam Optimization with Cosine Annealing

For finer control over the optimization, we employ the Adam optimizer [4] with decoupled weight decay (AdamW) and cosine annealing learning rate schedule [5]. Since the quantum expectation value $\langle H \rangle$ is not directly differentiable through the quantum circuit, the gradient is estimated via finite differences (Equation 53) at each iteration and supplied to the Adam update rule.

The Adam update rule maintains first and second moment estimates of the gradient:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (54)$$

where g_t is the gradient at step t , and β_1, β_2 are exponential decay rates for the first and second moments, respectively. We set $\beta_1 = 0.95$ (larger than the default 0.9) to increase the momentum window size, since the finite-difference gradients exhibit high fluctuation and require stronger smoothing. Conversely, we set $\beta_2 = 0.98$ (smaller than the default 0.999) to reduce the second moment window size, since the total training horizon is short (300 iterations) and the optimizer must adapt its step size more quickly. The parameters are updated as:

$$\theta_{t+1} = \theta_t - \eta_t \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (55)$$

where $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected estimates and η_t is the learning rate at step t .

The learning rate follows a cosine annealing schedule:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos \left(\frac{t}{T_{\max}} \pi \right) \right) \quad (56)$$

where η_0 is the initial learning rate, $\eta_{\min}$ is the minimum learning rate, and $T_{\max}$ is the maximum number of iterations.

The nominal learning rate is set to $\eta_0 = 0.01$ with $\eta_{\min} = 0.0003$. A weight decay of $\lambda_{\text{wd}} = 0.01$ is applied (AdamW formulation). Since the cost Hamiltonian is scaled by the boost factor α (Section 1.5), the gradients are also amplified by α . Consequently, the effective learning rate becomes $\eta_{\text{eff}} = \eta_0 \cdot \alpha$. For example, with $\alpha = 7500$, the effective learning rate is $0.01 \times 7500 = 75$.

The optimization terminates after $T_{\max} = 300$ iterations or when the cost function change falls below a tolerance $f_{\text{tol}} = 10^{-4}$ for 3 consecutive iterations.

1.7.3 Parameter Initialization

The initial parameters are scaled based on the Hamiltonian coefficients. Let $c_{\min}$ be the smallest absolute coefficient among all single-qubit, two-qubit, and mixer terms. The γ parameters are initialized uniformly in $[-\pi/c_{\min}, \pi/c_{\min}]$, while β parameters are initialized in $[-\pi, \pi]$. Alternatively, all parameters may be initialized to zero.

1.8 Solution Evaluation

After optimization, the final quantum state $|\gamma^*, \beta^*\rangle$ is used to evaluate solution quality.

1.8.1 Feasible Set Definition

We define the set of ε -feasible states:

$$\mathcal{F}_\varepsilon = \{\mathbf{b} \in \{0, 1\}^n \mid |\mathbf{P}'^\top \mathbf{b} - 1| \leq \varepsilon\} \quad (57)$$

The objective value for each basis state is computed without the penalty term:

$$f(\mathbf{b}) = \mathbf{b}^\top Q^{(\text{eval})} \mathbf{b}, \quad Q^{(\text{eval})} = Q|_{\lambda=0} \quad (58)$$

1.8.2 Approximation Ratio

Let $p(\mathbf{b}) = |\langle \mathbf{b} | \gamma^*, \beta^* \rangle|^2$ be the measurement probability of state $\mathbf{b}$. The expected objective value is:

$$\bar{f} = \sum_{\mathbf{b} \in \{0, 1\}^n} p(\mathbf{b}) \cdot f(\mathbf{b}) \quad (59)$$

The approximation ratio is defined relative to the feasible range:

$$r = \frac{\bar{f} - f_{\min}}{f_{\max} - f_{\min}} \quad (60)$$

where $f_{\min} = \min_{\mathbf{b} \in \mathcal{F}_\varepsilon} f(\mathbf{b})$ and $f_{\max} = \max_{\mathbf{b} \in \mathcal{F}_\varepsilon} f(\mathbf{b})$.

A value of $r = 1$ indicates the quantum solution matches the best feasible classical solution, while $r = 0$ corresponds to the worst feasible solution.

1.8.3 Max-Probability Ratio

Let $\mathbf{b}^* = \arg \max_{\mathbf{b}} p(\mathbf{b})$ be the most probable measurement outcome. The max-probability ratio evaluates the quality of this single state:

$$r_{\max} = \frac{f(\mathbf{b}^*) - f_{\min}}{f_{\max} - f_{\min}} \quad (61)$$

1.8.4 Additional Metrics

We also evaluate the optimized circuit against separate Hamiltonians to decompose the solution quality:

- **Budget Violation:** $\langle H_\lambda \rangle / \alpha$, where H_λ encodes only the penalty term $\lambda(\mathbf{P}'^\top \mathbf{b} - 1)^2$.
- **Expected Return:** $\langle H_{\text{ret}} \rangle / \alpha$, where H_{ret} encodes only $\boldsymbol{\mu}'^\top \mathbf{b}$.
- **Portfolio Risk:** $\langle H_{\text{risk}} \rangle / \alpha$, where H_{risk} encodes only $q \cdot \mathbf{b}^\top \Sigma'' \mathbf{b}$.

1.9 Barren Plateau Analysis

To investigate whether the QAOA cost landscape exhibits barren plateaus [7] as the number of qubits increases, we perform a random sampling analysis.

For a given problem instance with n qubits, we sample $N_s = 2000$ parameter vectors $\{(\boldsymbol{\gamma}^{(s)}, \boldsymbol{\beta}^{(s)})\}_{s=1}^{N_s}$ uniformly at random and evaluate the cost function:

$$E_s = \langle \boldsymbol{\gamma}^{(s)}, \boldsymbol{\beta}^{(s)} | H_C | \boldsymbol{\gamma}^{(s)}, \boldsymbol{\beta}^{(s)} \rangle \quad (62)$$

We compute the sample mean and variance:

$$\bar{E} = \frac{1}{N_s} \sum_{s=1}^{N_s} E_s, \quad \text{Var}(E) = \frac{1}{N_s - 1} \sum_{s=1}^{N_s} (E_s - \bar{E})^2 \quad (63)$$

A barren plateau is indicated when $\text{Var}(E) \rightarrow 0$ exponentially as n increases, meaning the cost landscape becomes flat and gradients vanish, making optimization intractable.

1.10 Experimental Configuration

Table 1 summarizes all hyperparameters used in the experiments. The quantum simulation is performed using the NVIDIA CUDA Quantum (CUDAQ) statevector simulator.

Table 1: Summary of hyperparameters.

Category	Parameter	Value
<i>Problem Setup</i>		
	Dataset	50 US equities
	Price filter $[P_{\min}, P_{\max}]$	$[108, 216]$
	Assets N	$\{3, 4, 5, 6, 7, 8\}$
	Qubits per asset d	2
	Total qubits $n = N \times d$	$\{6, 8, 10, 12, 14, 16\}$
	Experiments per setting E	10
	Budget violation tolerance ε	0.1
<i>QUBO & Ising Hamiltonian</i>		
	Risk-aversion q	1.5
	Budget penalty λ	0.0005
	Boost factor α	See Tables 2, 3
<i>QAOA Circuit</i>		
	Layers p	5
	Basis states K (Preserving mixer)	$\{12, 24\}$
<i>Adam Optimizer</i>		
	Learning rate η_0	0.01
	Minimum learning rate $\eta_{\min}$	0.0003
	Momentum β_1	0.95
	Second moment β_2	0.98
	Weight decay λ_{wd}	0.01
	Max iterations $T_{\max}$	300
	Convergence tolerance f_{tol}	10^{-4}
	Finite-difference shift δ	10^{-4}
	Early stopping patience	3 iterations
<i>Genetic Algorithm</i>		
	Population size N_{pop}	2000
	Generations G	35
	Crossover rate p_c	0.85
	Mutation rate p_m	$1.5/n$, where $n = N \times d$
	Tournament size	5
	Elitism count	2
<i>Barren Plateau Analysis</i>		
	Random samples N_s	2000
	Problem instances	10

The boost factor α varies across mixer types, number of assets, and penalty strength, and is summarized in Tables 2 and 3.

Table 2: Boost factor α for the X-mixer by number of assets N and budget penalty λ .

	$N = 3$	$N = 4$	$N = 5$	$N = 6$	$N = 7$	$N = 8$
$\lambda = 0.0005$	20000	10000	5000	2500	1800	1400
$\lambda = 0.005$	15000	7500	3750	2000	1250	800
$\lambda = 0.05$	1000	500	400	400	400	250
$\lambda = 0.5$	100	50	40	40	40	25
$\lambda = 5$	10	5	4	4	4	2.5

Table 3: Boost factor α for the Preserving mixer by number of assets N .

	$N = 3$	$N = 4$	$N = 5$	$N = 6$
α	30000	18000	10500	5250

References

- [1] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.
- [2] Franz G. Fuchs, Kjetil A. Lye, Halvor M. Nilsen, Alexander J. Stasik, and Joakim Sundnes. Efficient encoding of the weighted max-k-cut on a quantum computer using qaoa. *SN Computer Science*, 3(2):1–14, 2022. arXiv:2203.06095.
- [3] Stuart Hadfield, Zhihui Wang, Bryan O’Gorman, Eleanor G. Rieffel, Davide Venturelli, and Rupak Biswas. From the quantum approximate optimization algorithm to a quantum alternating operator ansatz. *Algorithms*, 12(2):34, 2019.
- [4] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.
- [5] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.
- [6] Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.
- [7] Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9(1):4812, 2018.